

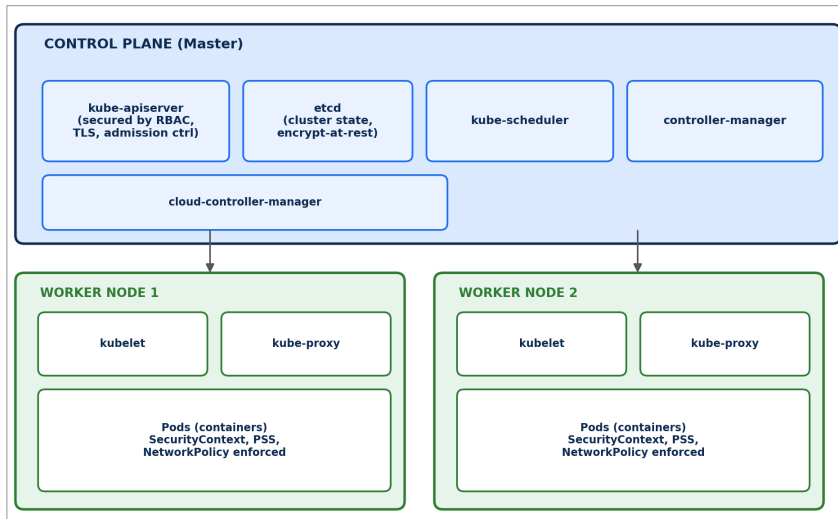


Figure 1: Kubernetes cluster architecture — control plane and worker nodes (Source: <https://kubernetes.io/docs/concepts/overview/components/>)



Figure 2: Public incident reporting by RedLock and Wired on the exposure of Tesla's Kubernetes Dashboard to the public internet (Source: <https://www.wired.com/story/cryptojacking-tesla-amazon-cloud/>)

a RoleBinding or ClusterRoleBinding attaches that set of permissions to a subject. The Principle of Least Privilege (PoLP) is the guiding rule: subjects — including CI/CD service accounts — should be granted only the permissions strictly required for their function, since granting cluster-admin broadly makes lateral movement trivial for an attacker who compromises any single credential.

Table 1 summarises the core RBAC objects and how they form an access-control decision.

The five objects in this table always work together as a chain, never in isolation. Consider a CI/CD pipeline that needs to deploy to the 'staging' namespace. The correct pattern is to create a dedicated ServiceAccount for the pipeline, define a narrowly scoped

Role that permits only 'create' and 'update' on Deployments and Services within 'staging', and connect the two with a RoleBinding — nothing more. A ClusterRole and ClusterRoleBinding would only be justified if the same pipeline genuinely needed to act across every namespace (for example, a cluster-wide monitoring agent). A well-documented real-world failure of this pattern is the legacy Helm 2 'Tiller' component: its simplest install path granted the Tiller ServiceAccount a full cluster-admin ClusterRoleBinding and exposed an unauthenticated in-cluster endpoint, meaning any pod that could reach Tiller could deploy arbitrary resources and inherit cluster-admin rights — exactly the over-broad-binding failure mode Table 1's least-privilege pattern is designed to prevent.

Network policies: By default, all pods in a Kubernetes cluster can communicate with all other pods — a flat network with no isolation. NetworkPolicy resources restrict this by defining allowed ingress/egress traffic based on pod and namespace labels. Current security checklists recommend starting with a default deny-all policy per namespace and then explicitly allowing only the traffic paths an application needs, combining this with RBAC and secrets management to build a defence-in-depth architecture.

Pod and container hardening: Pod Security Standards (PSS) define three levels of restriction that can be

Table 1: Core Kubernetes RBAC objects (Source: <https://kubernetes.io/docs/reference/access-control/rbac/>)

Object	Scope	Purpose	Example
Role	Namespace	Defines allowed verbs on resources within one namespace	Allow 'get,list' on pods in 'dev'
ClusterRole	Cluster-wide	Defines allowed verbs on resources across all namespaces, or on cluster-scoped resources	Allow 'list' on nodes
RoleBinding	Namespace	Grants a Role (or ClusterRole) to a subject within one namespace	Bind 'dev-viewer' Role to user 'alice'
ClusterRoleBinding	Cluster-wide	Grants a ClusterRole to a subject across the whole cluster	Bind 'cluster-admin' to a break-glass account
ServiceAccount	Namespace	Identity used by pods/CI pipelines to call the API server	CI pipeline deploying to 'staging'